



Containers, Máquinas Virtuais, Hosting e Colocation

Entendendo formas modernas de hospedar e executar aplicações — **UC8 · Arquitetura Cloud Computing e Big Data**

AULA PRESENCIAL · 3H

UC8

"Quando você acessa um site ou app, onde exatamente ele está rodando?"



Netflix

Milhões de streams simultâneos em servidores distribuídos globalmente



iFood

Pedidos em tempo real, integração com entregadores e restaurantes



Nubank

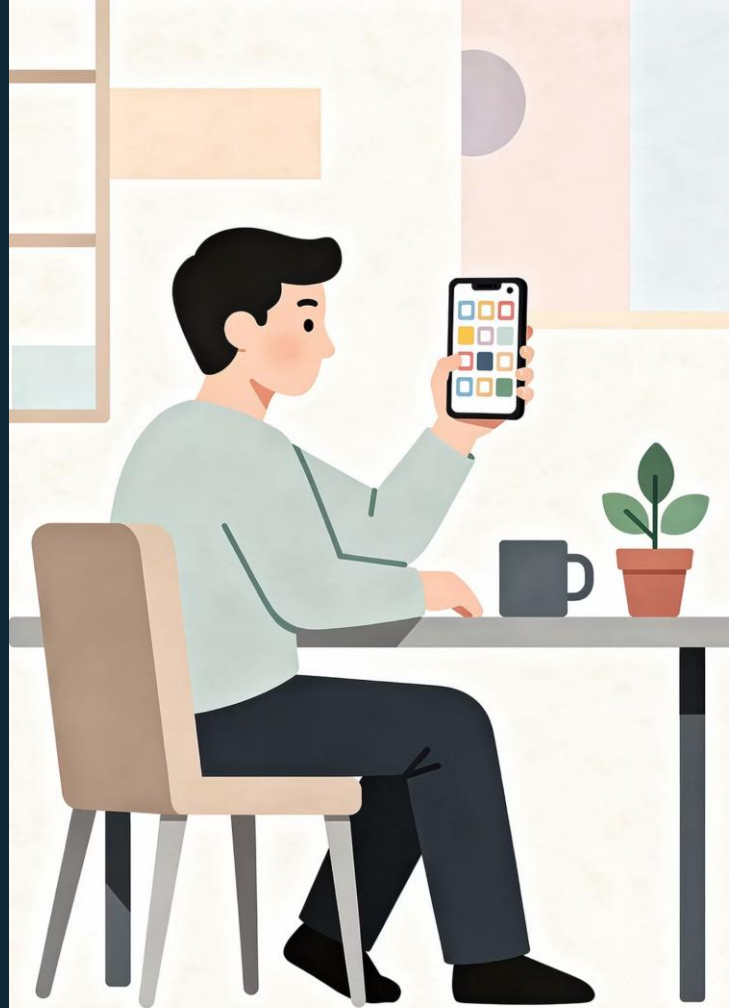
Transações financeiras com altíssima disponibilidade e segurança



App da Escola

Sistemas de notas, frequência e comunicação acessados por alunos e professores

Toda aplicação precisa de um **lugar para rodar**. Mas esse lugar pode ser muito diferente de um caso para outro.





O Problema: Antes de Publicar uma Aplicação...

Publicar uma aplicação vai muito além de escrever código. É preciso responder perguntas críticas de infraestrutura antes de colocar qualquer sistema no ar.

- **Onde ela vai rodar?**
Servidor próprio, nuvem, provedor de hospedagem?
- **Quem cuida do servidor?**
Sua equipe, um provedor terceirizado, ou um datacenter?
- **Como escalar?**
E se o tráfego triplicar de repente? A solução aguenta?
- **Como atualizar sem quebrar tudo?**
Deploys com zero downtime são um desafio real em produção.

O que é Hosting?

Hosting (hospedagem) é o serviço em que uma empresa disponibiliza espaço em seus servidores para que você publique sua aplicação ou site.

Você não precisa gerenciar o hardware — o provedor cuida disso. Você apenas sobe seu código e a aplicação fica acessível na internet.

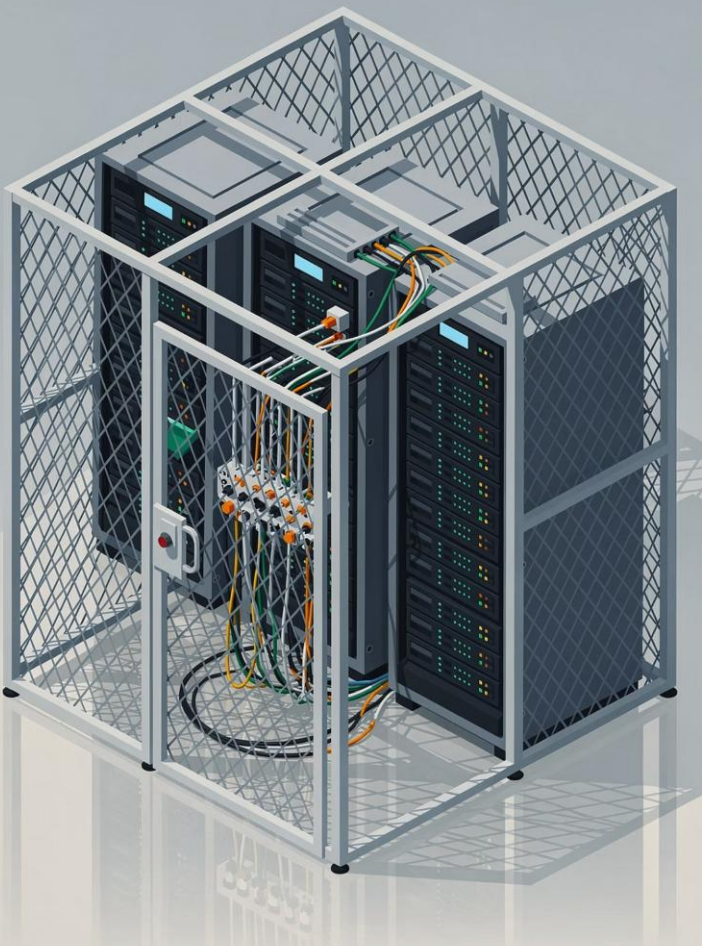
Exemplos práticos de uso

- Sites institucionais de empresas e escolas
- Blogs pessoais e portfólios
- Landing pages de marketing
- Sistemas web simples com baixo tráfego

Provedores populares

- Locaweb, HostGator, Umbler (Brasil)
- GoDaddy, Hostinger (global)

 Ideal para quem quer simplicidade, sem se preocupar com servidores.



Colocation: Seu Servidor, Datacenter Deles

Como funciona?

Na colocation, a **empresa possui seus próprios servidores físicos**, mas os instala dentro de um datacenter especializado. O datacenter fornece energia, resfriamento, segurança física e conectividade.



Você é responsável pelo hardware, sistema operacional e software.

Vantagens e Desafios



Vantagens

- Controle total sobre o hardware
- Melhor desempenho para workloads específicas
- Infraestrutura profissional de datacenter



Desafios

- Alto custo de equipamentos
- Equipe técnica especializada necessária
- Escalabilidade mais lenta e cara

Máquinas Virtuais: Um Computador Dentro do Computador



Uma **VM (Virtual Machine)** simula um computador completo dentro de um servidor físico. Cada VM tem seu próprio sistema operacional, memória, CPU, disco e rede — totalmente isolados das outras VMs. O **hypervisor** (como VMware, Hyper-V ou KVM) é o software que gerencia essa divisão de recursos.

Isolamento forte

Cada VM é totalmente independente

SO completo

Linux, Windows, qualquer sistema

Mais pesada

Boot lento, consome mais recursos

Containers: Leve, Rápido e Portátil



O que é um container?

Um container empacota a **aplicação + suas dependências** (bibliotecas, configurações, runtime) em uma unidade isolada — mas compartilha o **kernel do sistema operacional** do host.

Por que isso importa?

- **Leve:** não precisa de um SO completo por container
- **Rápido:** inicia em segundos, não minutos
- **Portátil:** roda igual em qualquer ambiente
- **Consistente:** elimina o "na minha máquina funciona"



Analogia: como uma caixinha de transporte que leva tudo que a aplicação precisa para funcionar em qualquer lugar.

Comparação: Hosting, Colocation, VM e Container

Solução	Controle	Custo	Isolamento	Velocidade	Portabilidade
 Hosting	Baixo	\$ Baixo	Compartilhado	 Instantâneo	Baixa
 Colocation	Alto	\$ \$ \$ Alto	Total (físico)	 Lento	Baixa
 VM	Alto	\$ \$ Médio	Forte (SO)	 Médio	Média
 Container	Médio	\$ Baixo	Processo	 Rápido	Alta

❏ Não existe uma solução "melhor" — a escolha depende do **cenário, custo e requisitos** de cada projeto.



"Se funciona no meu computador, por que não funciona no servidor?"

Esse é um dos problemas mais clássicos do desenvolvimento de software. A resposta quase sempre está em **ambientes diferentes**.

Dev usa

Python 3.11, biblioteca X versão 2.1,
variáveis locais

Servidor tem

Python 3.8, biblioteca X versão 1.9, outras
configs

Resultado

Erro misterioso em produção que nunca aparece localmente



Por que Containers Ficaram Tão Populares?



Consistência de Ambiente

Elimina o "na minha máquina funciona". O container carrega tudo que precisa para rodar igual em qualquer lugar.



Deploy Simplificado

Uma imagem, vários ambientes. Desenvolva, teste e suba para produção com o mesmo artefato.



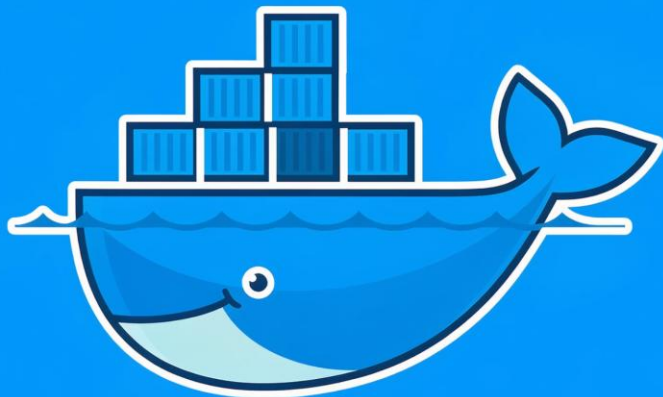
Escalabilidade Fácil

Suba dezenas de containers em segundos para absorver picos de demanda.



Microserviços

Cada parte da aplicação vive no seu próprio container — isolada, independente e escalável.



O que é Docker?

Docker é a plataforma mais popular para criar, executar e gerenciar containers. Lançado em 2013, revolucionou a forma como aplicações são empacotadas e distribuídas.

"Build once, run anywhere."

O que o Docker faz?

- Empacota a aplicação e todas as dependências
- Garante ambiente reproduzível em qualquer máquina
- Facilita o compartilhamento de imagens via Docker Hub
- Integra com ferramentas de CI/CD e orquestração

Onde é usado?

- Desenvolvimento local
- Pipelines de integração contínua
- Ambientes de produção em cloud
- Kubernetes e orquestração de containers



Docker está presente em Netflix, Spotify, Nubank e praticamente toda empresa de tecnologia moderna.

Imagem x Container: Qual a Diferença?

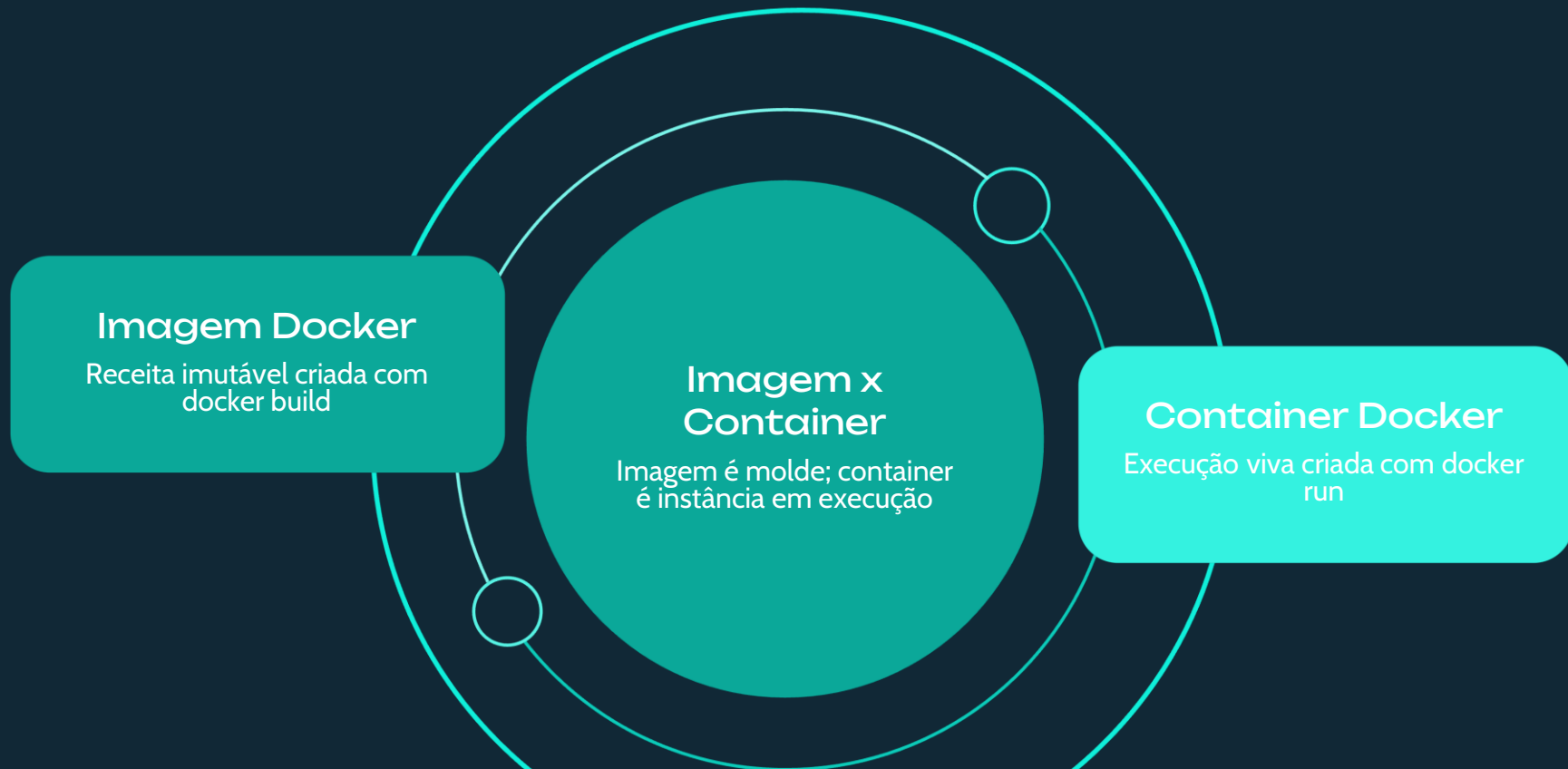
Imagem Docker

É o **modelo, o molde, a receita**. Define o sistema operacional base, dependências, código, variáveis e configurações. Uma imagem é *imutável*— não muda depois de criada.

Analogia: a **forma de bolo** ou a **receita escrita**.

Container Docker

É a **execução real da Imagem**. Quando você "roda" uma imagem, cria um container — a aplicação de fato em execução, com memória, processos e estado. Analogia: o **bolo já assado** — feito a partir da receita, mas agora existe de verdade.



Conceitos Essenciais do Docker

1

Dockerfile

Arquivo de texto com instruções para construir a imagem. Define SO base, dependências, arquivos copiados e comandos de inicialização.

2

Imagem

Modelo imutável gerado pelo Dockerfile. Pode ser armazenada e compartilhada no Docker Hub ou em registries privados.

3

Container

Instância em execução de uma imagem. Pode ser iniciado, parado, removido e recriado a qualquer momento.

4

Porta

Canal de comunicação para acessar a aplicação. Ex: `-p 8080:8000` mapeia porta externa para interna.

5

Volume

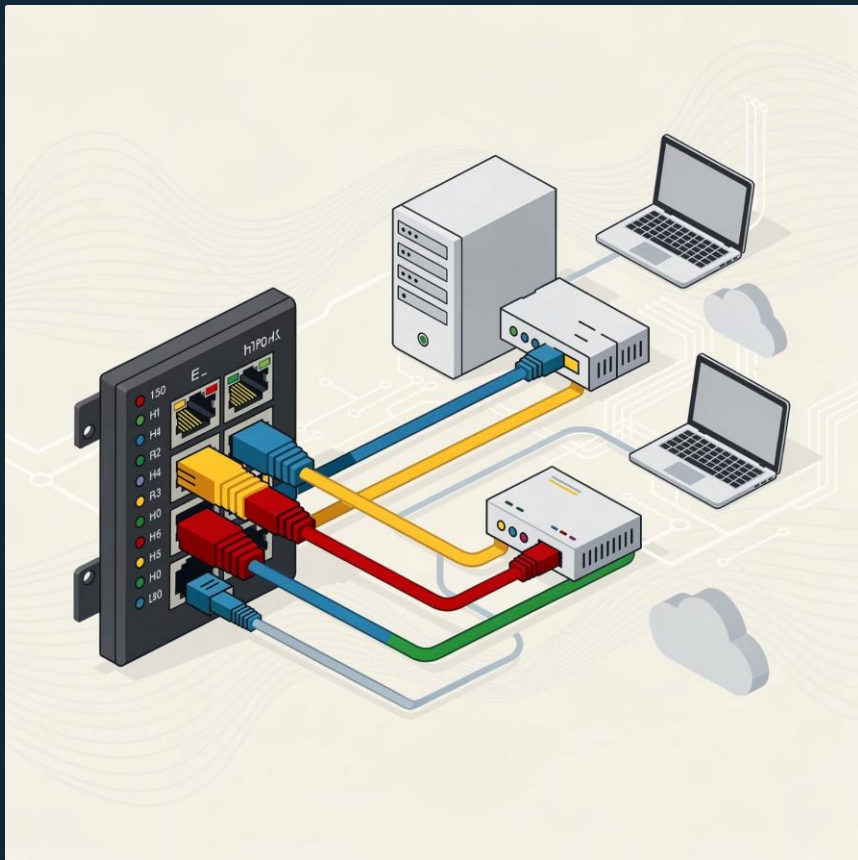
Mecanismo para persistir dados fora do container. Dados sobrevivem mesmo que o container seja removido.

6

Variável de Ambiente

Configurações externas ao código: senhas, tokens, URLs. Passadas na execução com `-e VARIÁVEL=valor`.

Porta: A Janela de Acesso ao Container



O que é uma porta?

A porta é o **canal de comunicação** pelo qual o mundo externo acessa a aplicação rodando dentro do container. Sem mapeamento de porta, o container fica "invisível" para o navegador ou qualquer cliente.

Exemplo prático

Aplicação Flask/FastAPI rodando internamente na porta **8000** do container.
Navegador acessando `localhost:8080` na máquina local.

O comando Docker: `-p 8080:8000`

- `8080` → porta do seu computador (externa)
- `8000` → porta dentro do container (interna)

Volume: Dados que Sobrevivem ao Container



Por que volumes são essenciais?

Containers são **efêmeros por natureza** — quando são removidos, tudo dentro deles some. Volumes resolvem isso ao armazenar dados **fora do container**, no sistema de arquivos do host.

Casos de uso comuns

- Banco de dados (PostgreSQL, MySQL, MongoDB)
- Arquivos enviados por usuários
- Logs de aplicação
- Configurações persistentes



Sem volume, dados de banco de dados são perdidos ao recriar o container!

Variáveis de Ambiente: Configuração Segura

O que são e por que usar?

Variáveis de ambiente guardam **configurações externas ao código**. Em vez de colocar senhas e tokens diretamente no código-fonte (péssima prática!), você passa essas informações na hora de executar o container.

Exemplos práticos



DB_PASSWORD

Senha do banco de dados



API_TOKEN

Token de autenticação



DB_URL

URL de conexão ao serviço



ENV

dev / staging / prod

Como usar no Docker

Na linha de comando:

```
docker run -e DB_PASSWORD=1234  
-e ENV=production  
minha-app
```

Ou com arquivo `.env`:

```
docker run --env-file .env  
minha-app
```



Nunca commite senhas no Git! Use variáveis de ambiente.

VM ou Container: Quando Usar Cada Um?



Use VM quando...

- Precisar de **isolamento forte** entre aplicações por segurança
- Precisar rodar **sistemas operacionais diferentes** no mesmo servidor
- Houver necessidade de um **ambiente completo** e dedicado
- Trabalhar com **sistemas legados** que exigem configuração específica
- O ambiente for regulado (banking, governo, saúde)



Use Container quando...

- Precisar de **deploy rápido** e frequente (CI/CD)
- Quiser **empacotar aplicações** com suas dependências
- Trabalhar com **microserviços** independentes
- Precisar **escalar com facilidade** horizontal
- Quiser consistência entre dev, staging e produção



Na prática, muitas arquiteturas usam **ambos**: VMs como hosts e containers rodando dentro delas.

O que construiremos na próxima semana:

Vamos containerizar uma aplicação do zero, passo a passo. Acompanhe cada etapa:

1

Criar a Aplicação

Uma API simples em Python (Flask ou FastAPI) com um endpoint `/health`

2

Escrever o Dockerfile

Definir imagem base, copiar arquivos, instalar dependências, expor porta

3

Gerar a Imagem

```
docker build -t minha-api:v1 .
```

4

Executar o Container

```
docker run -p 8080:8000 minha-api:v1
```

5

Testar no Navegador

Acessar `localhost:8080/health` e ver a resposta da API

6

Inspecionar e Explorar

```
docker ps, docker logs, docker exec -it
```

Atividade em Grupo: Qual a Melhor Solução?

Cada grupo analisa um cenário e indica a melhor solução: **hosting**, **colocation**, **VM** ou **container**. Justifique a escolha, vantagens, riscos e cuidados de segurança.

1

Site Institucional

Empresa pequena, baixo tráfego, site de apresentação

2

Sistema Bancário

Alto controle de infraestrutura, regulação, dados sensíveis

3

API de Startup

Crescimento imprevisível, precisa escalar rápido

4

Sistema Legado

Depende de SO e versões específicas de bibliotecas

5

Microserviços

Aplicação com múltiplos serviços independentes

6

Banco de Dados

Precisa persistir dados com segurança e alta disponibilidade

"Containers não substituem tudo — mas mudaram profundamente como aplicações modernas são criadas, publicadas e escaladas."

Reflexão Final para o Projeto UC8

"No projeto da sua arquitetura Cloud Computing e Big Data, quais partes poderiam ser executadas em containers? Quais precisariam de VMs? E o que poderia estar em hosting simples?"

Containers

APIs, microserviços, workers de processamento, pipelines de dados

VMs

Bancos de dados críticos, sistemas legados, ambientes isolados

Hosting/Cloud

Front-end estático, CDN, armazenamento de arquivos